

working of gym, is a slightly modified version of a script given by the official documentation for gym. For the code to execute successfully, the gym API and some other dependent packages must be installed by running the following three commands in the terminal: `pip3 install pygame`, `pip3 install gym`, and `pip3 install gym[box2d]`.

```
1. import gym
2. env = gym.make("MountainCar-v0",
   render_mode="human")
3. observation, info = env.reset( )
4. for _ in range(1000):
5.     observation, reward, terminated,
       truncated, info = env.step(env.
       action_space.sample( ))
6. if terminated or truncated:
7.     observation, info = env.reset( )
8. env.close( )
```

Now, let us briefly go through the code line by line. Line 1 imports the package gym, which provides a collection of environments for reinforcement learning experiments. Line 2 creates an instance of the *MountainCar-v0* environment, which simulates a car trying to climb a mountain. The parameter `render_mode` specifies that the environment should be rendered in *human* mode, meaning that the environment will be displayed on the screen for human observation. Other environments that can be used here include *CartPole-v1* (a control problem where the agent must balance a pole on top of a cart by moving the cart left or right) and *LunarLanderContinuous-v2* (a control problem where the agent must control a lunar lander to safely land on the moon's surface). You can replace Line 2 with the following lines of code to get the effect of these environments: `env = gym.make("CartPole-v1")` and `env = gym.make("LunarLander-v2", render_mode="human")`. Line 3 resets the environment to its initial state and returns an observation of that state. Line 4 sets up a loop that will run

1000 times. Line 5 takes a step in the environment by choosing an action at random from the action space. Line 6 checks whether the episode is over. If the episode is over, Line 7 resets the environment to its initial state and returns an observation of that state. Finally, Line 8 closes the environment. Overall, the above script runs a loop where it takes random actions in the *MountainCar-v0* environment for 1000 steps. Figure 1 shows the environment generated by the above Python script. Please note that in order to enable real learning, you should use RL algorithms like SARSA, Q-Learning, etc. It is important to note that the current example is a toy script and does not demonstrate any significant learning.

A comprehensive understanding of AI and machine learning requires a thorough understanding of RL and the gym/gymnasium library also. However, given the scope of this series, we cannot delve into the details of these topics. If you want to learn more about RL, I recommend referring the textbook 'Reinforcement Learning: An Introduction' by Sutton and Barto.

Classical AI techniques

Until now, our conversations about AI have solely focused on machine learning based on neural networks. However, it is important to consider classical techniques beyond machine learning and neural networks. But why do I use the term 'classical' techniques?

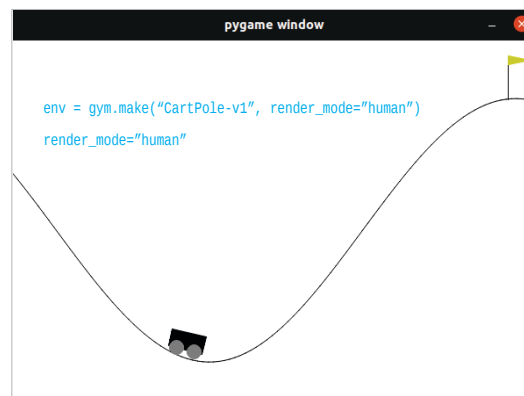


Figure 1: Output of the Python script using gym API

Well, the techniques we are about to discuss are often referred to as 'classical' because they have been used in AI since its inception, dating back to the early 1960s when processing power was limited to kilohertz and memory capacity was measured in kilobytes. In the early days of AI, there were differing opinions on how it should be implemented. One school of thought, known as 'connectionism', was responsible for the development of neural network based models of AI. The other school of thought, known as 'symbolic AI' or 'GOF AI (Good Old-Fashioned Artificial Intelligence)', included techniques that attempted to replicate the high-level conscious reasoning used by humans to solve problems. These techniques were quite successful in the initial stages of AI. Although machine learning and deep learning have become the dominant techniques in AI due to their success and the current availability of processing power and memory, it is important not to discount classical techniques, as they may regain relevance in the future.

Discussing non-machine learning based classical techniques of AI raises a dilemma about which techniques should be considered as part of AI. For example, could optimisation techniques, probabilistic reasoning based techniques such as Bayesian networks and heuristic search algorithms, and knowledge based systems

such as expert systems and rule based systems be considered AI techniques? While these techniques contain elements of AI, the question of what constitutes AI is a deeply philosophical and debated topic, and we are in no position to provide definitive answers. Therefore, our discussion is limited to logic based AI techniques and biologically inspired AI techniques.